

Laboration 3 – Sortering och iteratorer

Uppgift 1 – Iteratorer

Min implementation

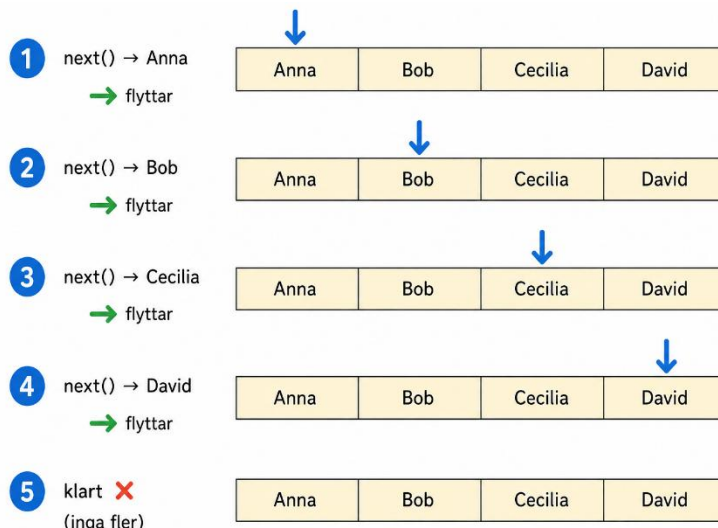
I denna laboration har jag vidareutvecklat min implementation av en länkad lista från laboration 1 genom att lägga till stöd för iteratorer. Detta gjordes genom att låta interfacet `List<T>` ärva från `Iterable<T>` samt implementera metoden `iterator()` i klassen `LinkedList`.

Iteratorn är implementerad som en privat inre klass i `LinkedList`. Den innehåller en referens till den aktuella noden i listan och används för att traversera listan steg för steg.

När iteratorn skapas pekar den på listans första nod, det vill säga head. Metoden `hasNext()` kontrollerar om det finns fler element genom att kontrollera om den aktuella noden är null eller inte. Metoden `next()` returnerar elementet i den aktuella noden och flyttar sedan referensen vidare till nästa nod.

Detta gör att listan kan användas i en for-each-loop istället för en vanlig for-loop, vilket ger en mer effektiv och tydlig traversal av listan.

Figur hur iteratorn fungerar



Figuren visar hur iteratorn börjar vid head och stegvis flyttar sig genom listan nod för nod tills den når slutet (null).

Hypotes

Jag tror att for-each-loopen kommer vara snabbare än en vanlig for-loop för en LinkedList. Anledningen är att for-each använder en iterator som går igenom listan direkt, medan en vanlig for-loop använder metoden `get(i)`, vilket innebär att listan måste traverseras från början varje gång ett element hämtas.

Resultat

Resultaten visar att for-each-loopen är snabbare än den vanliga for-loopen, särskilt när listans storlek ökar.

Analys

Den vanliga for-loopen använder metoden `get(i)`, vilket innebär att listan traverseras från början varje gång ett element ska hämtas. Detta leder till en hög tidsåtgång.

For-each-loopen använder i stället en iterator som går igenom listan en gång från början till slut, vilket gör den betydligt mer effektiv.

Resultaten stämmer därför väl överens med teorin.

Uppgift 2 – Sortering

Beskrivning av SortableList

Interfacet `SortableList` definierar metoder för att sortera en lista. Dessa metoder är:

- `bubbleSort()`
- `insertionSort()`
- `mergeSort()`
- `quickSort()`

Alla metoder returnerar en ny sorterad lista och ändrar inte original-listan.

Min implementation

Jag har implementerat klassen `SortableLinkedList`, som ärver från `LinkedList` och implementerar `SortableList`.

Klassen fungerar som en vanlig lista men innehåller även metoder för sortering. För att förenkla implementationen kopieras listans innehåll till en ArrayList, sorteras där och kopieras sedan tillbaka till en ny lista.

Detta val gjordes eftersom slumpmässig åtkomst är snabbare i en ArrayList än i en länkad lista, vilket gör sorteringsalgoritmerna enklare att implementera.

Svar på frågorna

Jag valde denna implementation eftersom den är enkel och tydlig. Genom att använda en ArrayList kan sorteringsalgoritmerna implementeras utan att behöva hantera länkade noder direkt.

Detta är en bra lösning ur ett pedagogiskt perspektiv, eftersom koden blir lättare att förstå. Däremot är det inte den mest effektiva lösningen, eftersom data kopieras mellan strukturer.

Ett möjligt användningsområde för denna implementation är system där data lagras dynamiskt men ibland behöver sorteras, till exempel listor med användare eller geografiska objekt.

Hur QuickSort fungerar

QuickSort använder ett pivot-element för att dela upp listan.

I min implementation används det sista elementet som pivot. Alla element som är mindre än pivot flyttas till vänster och alla större till höger. Därefter placeras pivot på sin slutliga position i listan.

Denna process upprepas rekursivt för de två delarna tills hela listan är sorterad.

Hypotes

Jag tror att Merge Sort och Quick Sort kommer vara snabbare än Bubble Sort och Insertion Sort för större listor.

Resultat

Resultaten visar att Bubble Sort och Insertion Sort är långsamma, medan Merge Sort och Quick Sort är betydligt snabbare.

Analys

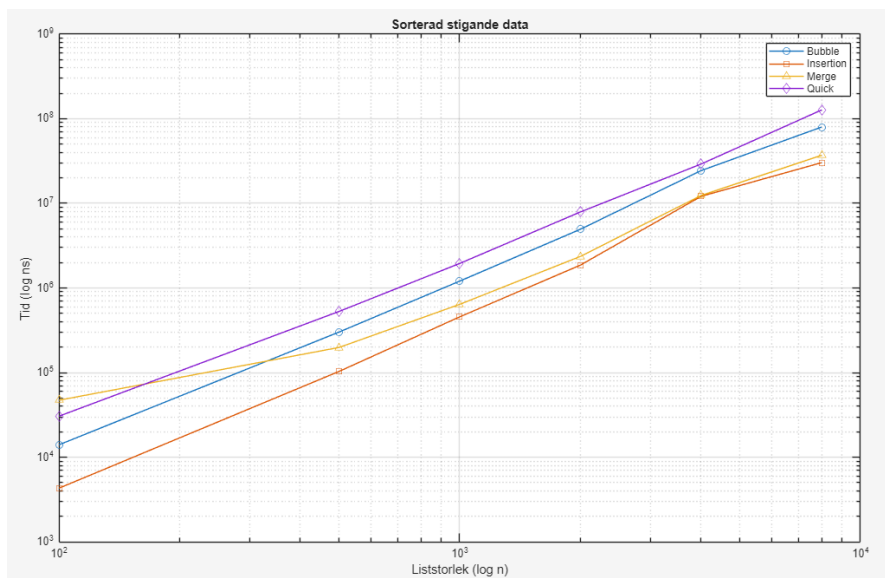
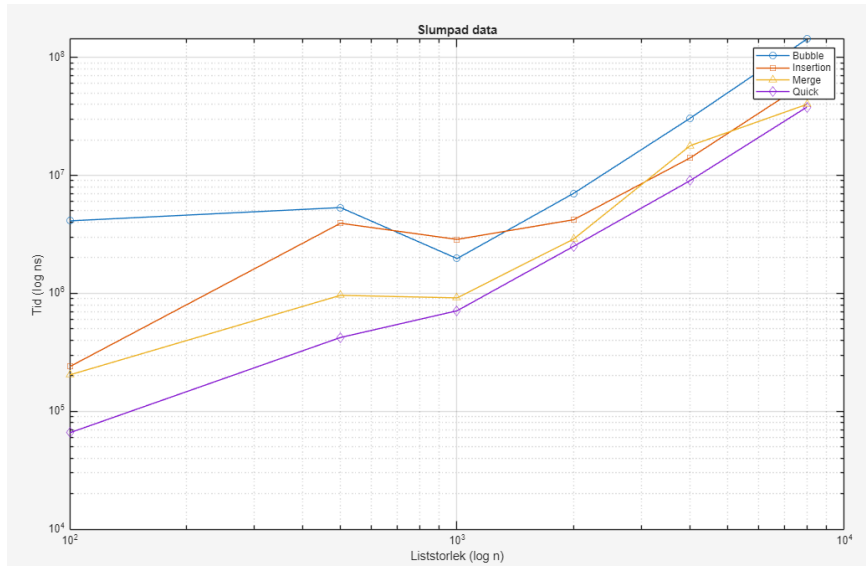
Skillnaden beror på algoritmernas tidskomplexitet. Bubble Sort och Insertion Sort har tidskomplexiteten $O(n^2)$, vilket gör att de blir ineffektiva för stora listor.

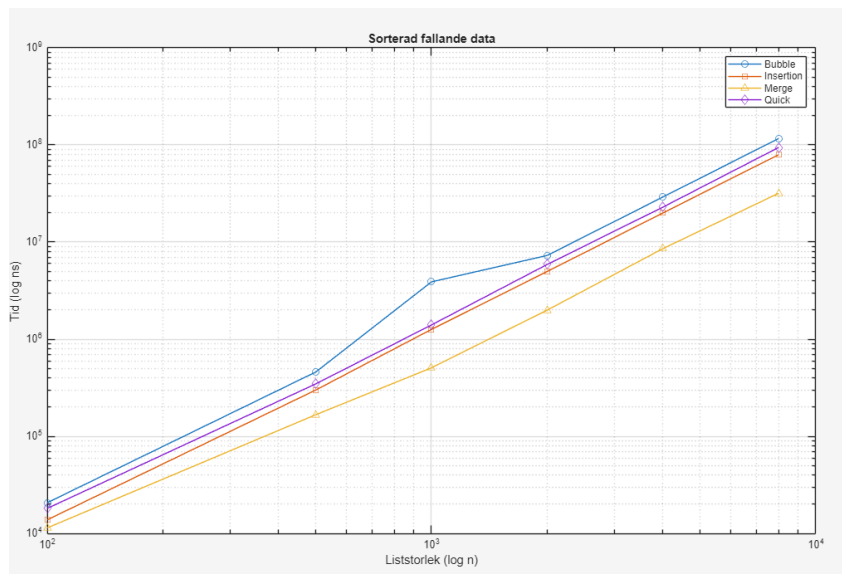
Merge Sort och Quick Sort har i genomsnitt tidskomplexiteten $O(n \log n)$, vilket gör dem betydligt mer effektiva.

Quick Sort kan dock bli långsammare i vissa fall beroende på pivot-val, till exempel om listan redan är sorterad.

Uppgift 3 Resultat och analys

Jag genomförde tidstester på tre olika typer av listor: slumpad data, redan sorterad stigande data och sorterad fallande data. Resultaten redovisades i tre log-log-grafer, en graf för varje typ av data.





Resultaten visar att Bubble Sort och Insertion Sort generellt får högre tidsåtgång när listans storlek ökar. Detta stämmer med att dessa algoritmer har tidskomplexiteten $O(n^2)$. I graferna syns detta genom att kurvorna ökar snabbare än för Merge Sort och Quick Sort.

Merge Sort har mer stabil prestanda i alla tre fallen. Den påverkas inte lika mycket av om datan är slumpad, sorterad eller fallande. Detta stämmer med teorin eftersom Merge Sort har tidskomplexiteten $O(n \log n)$.

Quick Sort är snabb på slumpad data, men blir tydligt långsammare på redan sorterad stigande data. Detta beror på att min implementation väljer sista elementet som pivot. När listan redan är sorterad blir uppdelningen obalanserad, vilket gör att Quick Sort närmar sig sitt sämsta fall, $O(n^2)$.

Insertion Sort fungerar bättre på sorterad stigande data än på slumpad eller fallande data, eftersom få element behöver flyttas när listan redan är i rätt ordning. På fallande data behöver många element flyttas, vilket gör den långsammare.

Sammanfattningsvis visar resultaten att datans ordning påverkar algoritmerna olika. Merge Sort är mest stabil, Quick Sort påverkas mycket av pivot-valet, och Bubble Sort samt Insertion Sort blir ineffektiva vid större datamängder.

Resultat

Resultaten från grafen visar tydliga skillnader mellan algoritmerna.

Bubble Sort och Insertion Sort ökar snabbt i tidsåtgång när listans storlek växer, medan Merge Sort och Quick Sort ökar betydligt långsammare.

För små listor är skillnaden mellan algoritmerna mindre, men när listans storlek ökar blir skillnaden mycket tydlig.

Analys

Resultaten stämmer väl överens med teorin om tidskomplexitet.

Bubble Sort och Insertion Sort har tidskomplexiteten $O(n^2)$, vilket innebär att tiden ökar kraftigt när listans storlek ökar. Detta syns i grafen som en brant lutning.

Merge Sort och Quick Sort har i genomsnitt tidskomplexiteten $O(n \log n)$, vilket gör att deras tidsökning är betydligt långsammare. I grafen syns detta som en flackare kurva.

Quick Sort påverkas av pivot-valet och kan i vissa fall prestera sämre, till exempel om listan redan är sorterad. Merge Sort har däremot stabil prestanda oberoende av datans struktur.

Sammanfattningsvis visar resultaten att mer avancerade algoritmer är betydligt mer effektiva för stora datamängder.

Slutsats

Laborationen gav en djupare förståelse för hur iteratorer fungerar och hur olika sorteringsalgoritmer skiljer sig åt.

Det blev tydligt att iteratorer är ett effektivt sätt att traversera länkade listor och att valet av sorteringsalgoritm har stor betydelse för prestandan.